



Universidad de Las Palmas de Gran Canaria

Faculty of Computer Science and Engineering

Task 4: Distributed Matrix Multiplication & Frequent Item Sets

Big Data – Academic Year 2025–2026

Individual Assignments

Author:

Adrián Ojeda Viera (DNI: 45364780V)

Repository:

<https://github.com/adriaanojeda/BIGDATA.git>

Submission Date: December 20, 2025

Comparison of distributed computing paradigms (Apache Spark) versus sequential algorithms, including an implementation of the Frequent Item Set problem using MapReduce logic.

Abstract

This report documents the design, implementation, and evaluation of distributed algorithms for large-scale data processing. The primary task focuses on **Matrix Multiplication** using Apache Spark (Java), comparing its performance against a sequential baseline. Results indicate that while distributed systems introduce significant overhead for small datasets ($N \leq 1000$), they are essential for scalability. Additionally, the optional **Frequent Item Set** problem was implemented using a native MapReduce approach in Python, successfully identifying purchasing patterns in a transactional dataset.

Contents

1	Introduction and Objectives	3
2	Methodology	3
2.1	Hardware & Software Environment	3
2.2	Strategy	4
3	Part I: Distributed Matrix Multiplication	4
3.1	Theoretical Approach	4
3.2	Implementation (Java + Spark)	4
3.3	Experimental Results	5
3.4	Discussion Analysis	5
3.4.1	The Cost of Distribution	5
3.4.2	Scalability Trend	5
4	Part II: Frequent Item Set (Optional Task)	6
4.1	Problem Definition	6
4.2	MapReduce Implementation (Python)	6
4.3	Results	6
5	Conclusion	7

1. Introduction and Objectives

In the era of Big Data, single-machine processing is often insufficient due to memory constraints and processing time. Distributed computing frameworks allows us to scale horizontally. This assignment covers two fundamental problems in data engineering:

1. **Distributed Matrix Multiplication:** A mathematical operation fundamental to machine learning ($C = A \times B$), which requires careful data partitioning when matrices exceed RAM capacity.
2. **Frequent Item Set Mining (Optional):** A classic data mining problem (market basket analysis) used to find associations between items (e.g., "People who buy milk also buy bread").

The objectives are:

- To implement a scalable matrix multiplication using **Apache Spark** (Java).
- To benchmark the distributed solution against a traditional sequential approach.
- To demonstrate the logic of **MapReduce** by solving the Frequent Item Set problem.

2. Methodology

2.1. Hardware & Software Environment

The benchmarks were conducted in a local simulation environment using the following specifications:

- **Processor:** AMD Ryzen 5 3600X (6 Cores / 12 Threads).
- **Memory:** 32 GB RAM.
- **Frameworks:**
 - **Java:** OpenJDK 21.0.9 (for Spark Implementation).
 - **Spark:** Apache Spark 3.5.0 (Core SQL).
 - **Python:** Version 3.13 (for MapReduce Logic).
- **Build Tool:** Maven (for dependency management).

2.2. Strategy

- **Main Task:** Spark was chosen over Hadoop MapReduce due to its in-memory processing capabilities, which significantly reduce disk I/O latency for iterative operations like matrix multiplication.
- **Optional Task:** A native Python script was developed to simulate the Map/Shuffle/Reduce phases without the overhead of a full Hadoop cluster, demonstrating algorithmic understanding.

3. Part I: Distributed Matrix Multiplication

3.1. Theoretical Approach

Given two matrices A ($m \times n$) and B ($n \times p$), the element C_{ij} is the dot product of row i of A and column j of B .

In a distributed environment, partitioning is key. We utilized the **Broadcast Variable Strategy**:

1. Matrix A is large and distributed row-by-row across the cluster (RDD).
2. Matrix B is assumed to fit in the memory of each worker node. It is **broadcasted** (sent once) to all nodes.
3. Each node computes the multiplication of its assigned rows of A against the complete matrix B .

3.2. Implementation (Java + Spark)

The core logic utilizes Spark's `parallelize` to distribute Matrix A and `broadcast` to share Matrix B.

```
1 // 1. Distribute Matrix A across the cluster
2 List<double[]> listA = generateDistributedMatrix(N, N);
3 JavaRDD<double[]> rddA = sc.parallelize(listA);
4
5 // 2. Broadcast Matrix B (Efficient network transfer)
6 double[][] matrixB = generateMatrix(N, N);
7 Broadcast<double[][]> broadcastB = sc.broadcast(matrixB);
8
9 // 3. Parallel Computation (Map Phase)
10 JavaRDD<double[]> rddC = rddA.map(rowA ->
11     // Each worker multiplies its row by the broadcasted B
12     multiplyRowByMatrix(rowA, broadcastB.value())
13 );
14
15 // 4. Trigger Execution
16 long count = rddC.count();
```

Listing 1: Spark Implementation using RDDs

3.3. Experimental Results

We compared the Spark implementation against a standard triple-loop Java implementation (‘BasicMatrixMultiplication’). The tests were run with random dense matrices of sizes $N = 100, 500, 1000$.

Table 1: Execution Time Comparison (Seconds)

Matrix Size ($N \times N$)	Sequential (Java)	Distributed (Spark)	Overhead Ratio
100×100	0.004 s	0.469 s	$\sim 117x$
500×500	0.045 s	0.610 s	$\sim 13x$
1000×1000	0.157 s	0.895 s	$\sim 5.7x$

3.4. Discussion Analysis

3.4.1 The Cost of Distribution

The results clearly show that for these matrix sizes, the **Sequential** version is significantly faster. This is expected and highlights the "Spark Overhead":

- **Startup Latency:** Spark requires time to initialize the **SparkContext**, negotiate resources with the JVM, and build the DAG (Directed Acyclic Graph) of tasks.
- **Serialization:** Moving data from the Driver to Executors (even locally) involves serializing Java objects, which is CPU intensive.

3.4.2 Scalability Trend

Despite the initial slowness, the **Overhead Ratio** drops dramatically as N increases (from $117x$ down to $5.7x$).

- In the sequential version, time grows strictly cubically ($O(N^3)$).
- In the Spark version, the parallel processing capability of the multi-core CPU (‘local[*]’) begins to pay off.
- **Critical Conclusion:** If we increased N to 10,000 or 20,000, the Sequential version would crash (OutOfMemory or excessive time), whereas Spark would succeed by spilling data to disk if necessary.

4. Part II: Frequent Item Set (Optional Task)

4.1. Problem Definition

The goal is to identify pairs of items that appear together in transactions more often than a minimum support threshold. This is the foundation of the **A-Priori algorithm**.

4.2. MapReduce Implementation (Python)

We implemented a native MapReduce simulation without external libraries to demonstrate the raw logic.

1. The Mapper: Reads a transaction (line) and emits every possible pair of items found in that transaction with a count of 1.

```
1 def mapper(line):
2     items = line.strip().split(',')
3     items.sort() # Sort to ensure (A,B) is same as (B,A)
4     emitted = []
5     for pair in combinations(items, 2):
6         emitted.append((pair, 1))
7     return emitted
```

2. The Shuffle (Sort): In our simulation, a dictionary collects lists of values for each key: Key: ('Item_1', 'Item_2') -> Values: [1, 1, 1, 1]

3. The Reducer: Sums the counts. If the sum meets the threshold, it is emitted.

```
1 def reducer(key, counts):
2     total = sum(counts)
3     if total >= MIN_SUPPORT:
4         return total
```

4.3. Results

The algorithm was tested with a generated dataset of 1,000 transactions containing 50 unique items. **Sample Output:**

```
['Item_1', 'Item_18']    11
['Item_15', 'Item_47']   11
['Item_20', 'Item_24']   10
['Item_28', 'Item_29']   10
...
```

The result correctly identified pairs that appeared together frequently (e.g., Item 1 and Item 18 appeared 11 times), validating the MapReduce logic.

5. Conclusion

This assignment successfully demonstrated the trade-offs in Distributed Computing:

1. **Overhead vs. Scalability:** For small tasks ($N < 1000$), distributed frameworks like Spark incur heavy overhead (Context initialization, Serialization) that makes them slower than simple Java loops. However, they provide the infrastructure necessary to handle Big Data workloads that exceed single-machine capacity.
2. **MapReduce Versatility:** The optional task proved that the MapReduce paradigm is effective for pattern mining problems like Frequent Item Sets, allowing complex global patterns to be discovered through simple local counting operations.
3. **Implementation Success:** Both the robust Spark implementation (via Maven) and the algorithmic Python simulation functioned correctly, producing consistent and verifiable results.